BACKEND TECHNICAL REFERENCE

FATHOM

An autonomous blockchain investigation engine. Given one address, it finds the nearest exchange the money came from and the nearest exchange it went to — and shows its working for every step.

Version 0.1.0 **Branch** feat/backend-v1 **Chains** 5 **Tables** 12 **Labels** 74,928

Tests 352 passing **Compiled** 13 Aug 2026

CONTENTS

The question it answers

An investigator holds one address from a report — a wallet in a fraud complaint, a ransom payment, a seized device. Two questions decide whether the case can progress.

`BACKWARD` **Where did this money come from?** Trace the funding history until it reaches a regulated exchange. That exchange holds KYC records on whoever deposited the funds — a name, an ID document, a bank account. It is a subpoena target.

`FORWARD` **Where did this money go?** Trace the cash-out until it reaches a regulated exchange. That exchange can freeze the account before the funds leave the system.

Both answers are only useful if they name a real, contactable business. So the engine reports two things per direction, never merged: the *nearest* endpoint it reached, and the nearest endpoint it could actually *name* from a citable source. When those are two different addresses they stay two rows, because a same-distance address the tool is only 62% sure about is information the investigator is entitled to see — not something to hide behind a confident-looking label.

The term of art is **VASP** — Virtual Asset Service Provider, the FATF category covering custodial exchanges, brokers and custodians. Binance, Coinbase, OKX and Kraken are VASPs. A Uniswap router is not: it holds nobody's funds and can freeze nothing. Confusing the two is the single most damaging error this system could make, and most of its design exists to prevent it.

> **What this system is not.** It is decision support, not a chain of custody. It makes no claim of court admissibility or legal certification. It does not de-anonymise mixers — when a trail enters Tornado Cash the finding says the trail was *cut*, not that it ran dry. It contains no machine learning: every conclusion is either a fact on the ledger, a named and versioned algorithm, a dated third-party claim, or a statement about the tool's own run.

The evidence model

Everything else in this document rests on one decision: a conclusion must declare what *kind* of support it has, and the four kinds are never conflated.

EVIDENCE KIND	MEANS	MUST CARRY	MUST NOT CARRY
`onchain_fact`	Anyone can verify this on a block explorer.	Transaction references	Confidence, source, heuristic
`heuristic_inference`	A named algorithm concluded this from behaviour.	`name@version`, confidence < 1.0	A source
`third_party_claim`	An outside source asserts this. Only this kind can name an operator.	Source, source date, confidence < 1.0	A heuristic
`engine_observation`	A statement about FATHOM's own run — what it examined, where it stopped.	—	Confidence, source, heuristic

These rules are enforced in the constructor of a frozen dataclass (`Evidence.__post_init__`), so there is no code path anywhere in the system that produces evidence which skipped the check. An ill-formed piece of evidence cannot be constructed, let alone stored.

> **Confidence is never 1.0 for an inference or a claim.**
>
> WHY — A confidence of 1.0 on a sourced label is a claim of ground truth. Every label is a claim, not truth — including a signature-verified one, because the signature proves key control, not that the operator is who they say they are.
>
> INSTEAD OF — One uniform optional confidence in [0, 1] applied to every kind.
>
> WHAT BROKE — The looser version shipped. A label dataset asserting `confidence == 1.0` crashed a live run mid-trace. The bound now sits in three places: the domain constructor, the attribution result, and a database CHECK.

A third-party claim may not carry a heuristic, and an inference may not carry a source.

WHY — The laundering guard. Without it, an inference can wear claim clothing with its heuristic still attached, and every consumer that switches on the evidence kind reads a guess as sourced attribution.

INSTEAD OF — One composite piece of evidence carrying both the source it rests on and the heuristic that processed it.

WHAT BREAKS — A reviewer reading "labelled by OFAC SDN, sweep@1, 0.7" cannot tell whether the address is on a sanctions list or whether a pattern detector merely thinks it resembles one — and cannot challenge the label's date separately from the algorithm's version.

A fourth kind, `engine_observation`, exists to *narrow* what `onchain_fact` means.

WHY — "The trail ends here" is a statement about the tool, verifiable against the investigation record and not against any ledger.

INSTEAD OF — Three kinds, filing tool state as an on-chain fact with the subject address as its reference.

WHAT BROKE — That shipped. A reviewer auditing the finding would follow the reference to a block explorer, find a real address, and conclude that "the frontier ran dry" had been independently verified on-chain. A regression test now carries the name `test_terminal_findings_never_wear_the_onchain_fact_stamp`.

One consequence is worth stating plainly to any reviewer, because the whole label pipeline is built around it: the function that decides whether an answer is *named* keys on the presence of `third_party_claim` evidence. That is the structural reason an unverified community report must never reach the attributor at any confidence — it would silently become a named answer.

Architecture

Seven layers, one dependency direction, and one hard boundary: only the chain adapters may touch the outside world.

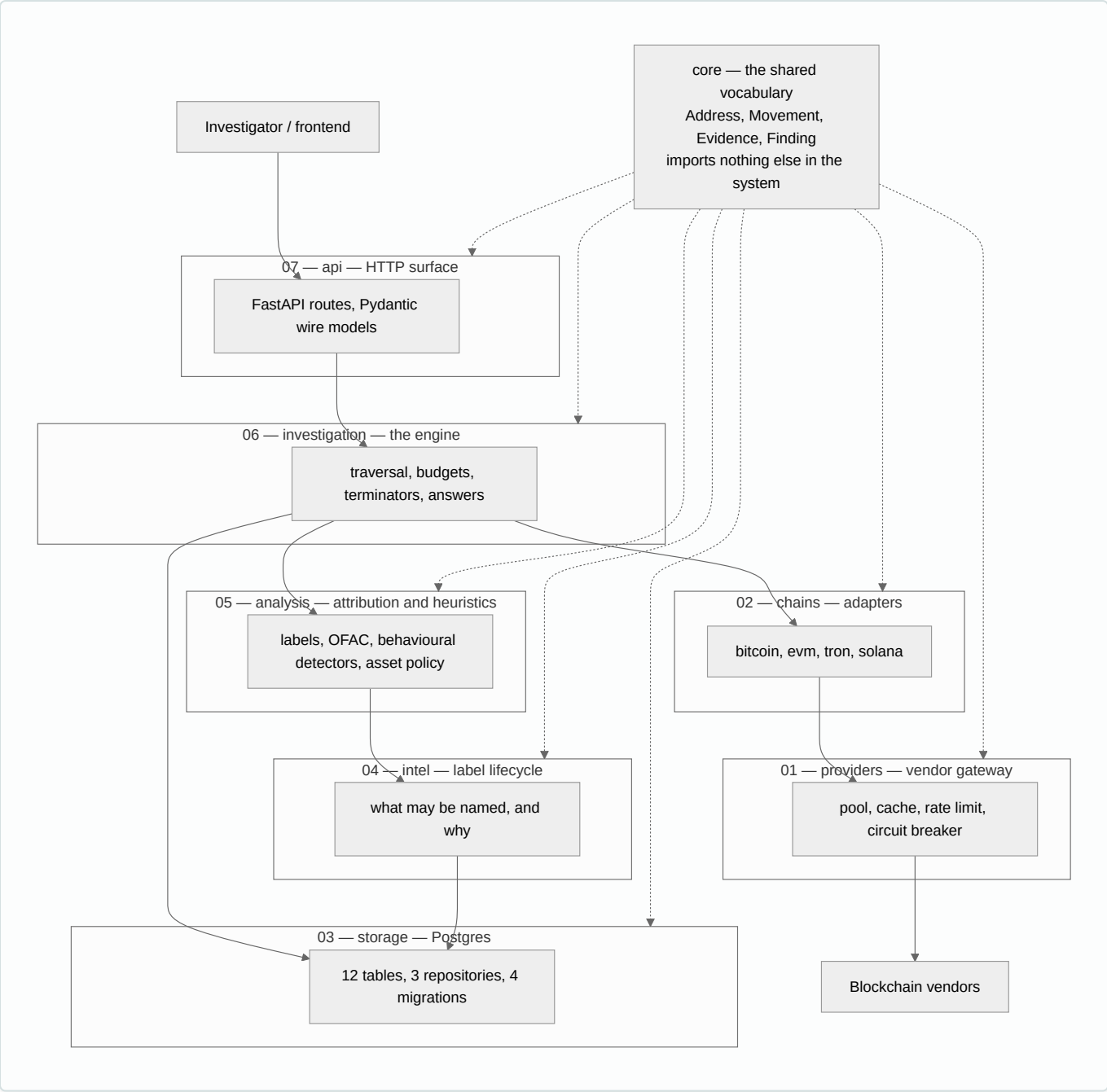

FIG 1 The dependency rule runs one way. The engine holds no reference to the provider pool at all — it physically cannot make a network call. `core` is imported by everything and imports nothing, so the vocabulary cannot drift.

LAYER	OWNS	DELIBERATELY DOES NOT OWN	LINES
`core`	The domain vocabulary and the evidence taxonomy	Any I/O, any judgement	712
`providers`	Which vendor answers, and at what cost	What the data means	~1,100
`chains`	Turning each ledger's semantics into one shape	Cross-chain reasoning, vendor choice	1,728
`storage`	Rows, constraints, migrations	Policy of any kind	1,624
`intel`	Which claims may become citable evidence	Reading labels during a trace	418
`analysis`	Naming, sanctions screening, behaviour	Traversal decisions	~1,200
`investigation`	Where to walk, when to stop, what the answer is	Network access, label meaning	1,360
`api`	Validation, wire shapes, chain disambiguation	Any investigation behaviour	715

01

Providers — the vendor gateway

The only route to any blockchain data source. Nothing above this layer knows which vendor answered a question.

FATHOM reads five chains through five vendor clients on free API tiers. That constraint drives the whole design: a quota spent twice on the same transaction is a bug, and one vendor's outage must not become "this address has no history".

A read enters as a vendor-neutral request — `(chain, capability, params)` — and the pool decides everything else. Capabilities are a closed set of nine (`address_history`, `tx_lookup`, `internal_traces`, `token_transfers`, and so on), so a caller asks for what it needs, never for a vendor.

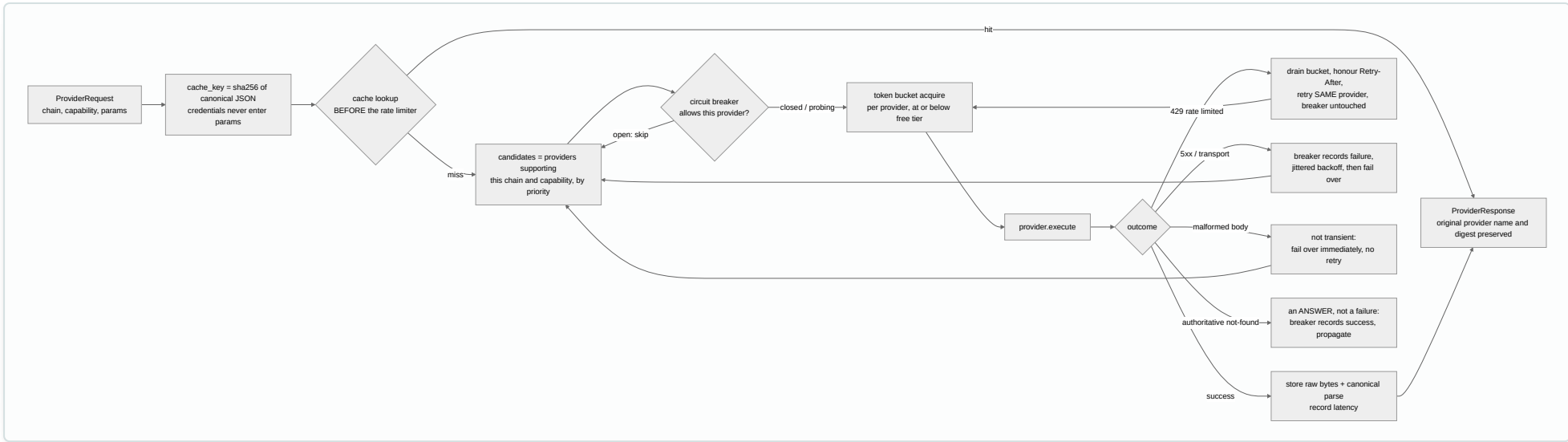

FIG 2 The four-class error taxonomy is the point. Each vendor client must translate its own quirks into these four classes, and each class gets a different policy.

The cache is consulted *before* the rate limiter, and the cache key never includes the provider.

WHY	Chain data is immutable, so a cache hit must cost zero rate budget. A vendor-independent key means any provider's answer can serve any other provider's identical request.
INSTEAD OF	Limiter first, or a provider-scoped key.
WHAT BREAKS	Etherscan is registered at 4 requests/second and mempool.space at 2. Failing over to a second vendor would re-pay for a transaction the pool already holds.

A rate-limit response retries the same provider and never counts against its circuit breaker.

WHY	A 429 means the vendor is alive and our configured rate was wrong — evidence about us, not about them.
INSTEAD OF	Treating every non-2xx as one retryable failure.
WHAT BREAKS	The provider you rely on most is the one you hit limits on first, so the breaker would preferentially evict your best source.

An authoritative "no such transaction" is recorded as a *success* and propagated without failover.

WHY	The provider worked fine. A declared absence is an answer.
INSTEAD OF	Treating a null or 404 as a failure and trying the next vendor.
WHAT BREAKS	Every unknown hash would cost up to (candidates × attempts) vendor calls and end in an infrastructure error — turning a truthful "this transaction does not exist on this chain" into a false outage report.

Credentials attach inside the vendor client at execute time, never in the request parameters.

WHY — Request parameters are hashed into the cache key and persisted in the evidence store.

INSTEAD OF — Passing the configured credential through as a normal parameter.

WHAT BREAKS — Rotating a key would silently invalidate the entire cache, and the key itself would be written verbatim into the database — a durable secret leak inside the evidence store. A related leak (HTTP client libraries logging full URLs with keys in them) was found in review and closed by raising those loggers to WARNING before anything else initialises.

CONFIGURED LIMITS, AS REGISTERED

VENDOR CLIENT	SERVES	RATE	CAPABILITIES	CREDENTIAL
`MempoolSpaceProvider`	Bitcoin	2 / s	4	None needed
`EtherscanV2Provider`	Ethereum, Polygon	4 / s	3	Required to register
`EvmRpcProvider`	Ethereum, Polygon	5 / s	5	One instance per configured endpoint
`SolanaRpcProvider`	Solana	5 / s	4	One instance per configured endpoint
`TronGridProvider`	Tron	5 / s	3	Optional; drops to 2/s without one

Nine RPC endpoint templates span three chains. A vendor is registered only if its credential is set, so an absent key removes that vendor silently rather than failing startup — one expired key must not take the platform down when three other vendors serve the same capability. With no keys at all the pool raises a structured "all providers failed, 0 attempts" rather than returning empty data.

02

Chain adapters — one shape for four paradigms

Bitcoin, Ethereum, Polygon, Tron and Solana keep value in fundamentally different ways. Everything above this layer sees one.

The atomic traced fact is a **Movement**: an amount, an asset, a transaction, a timestamp, and up to two endpoints. Both endpoints may not exist — that is what makes one shape cover every ledger.

CHAIN	MODEL	HOW VALUE IS READ	ADDRESS FORM
Bitcoin	UTXO	Each spending input becomes a half with no recipient; each paying output a half with no sender. The transaction joins them.	Base58 or bech32 — case preserved
Ethereum · Polygon	Account	Three feeds merged: normal transactions, token transfers, and internal traces.	Hex, lowercased
Tron	Account	Native and TRC-20 feeds merged; hex `41…` converted to Base58 `T…` in-process.	Base58 — case preserved
Solana	Balance delta	No transfers exist to read. Accounts whose balance fell are inputs; accounts whose balance rose are outputs; the fee is added back for the payer.	Base58 pubkey — case significant

Solana's balance-delta model is mapped onto the existing UTXO halves rather than getting its own movement kind.

WHY — Counterparty resolution then stays data-driven, and the engine never branches on chain identity.

INSTEAD OF — A Solana-specific movement kind, or letting the engine special-case Solana.

WHAT IT BOUGHT — Adding a third execution paradigm required *zero* changes to the engine, the heuristics, the graph queries or the report.

Ethereum reads a third feed — internal traces — that most tools skip.

WHY — Contract-delivered native value appears in neither of the other two feeds. Mixer withdrawals, exchange withdrawal contracts, smart-contract wallets and bridge exits are all invisible without it.

INSTEAD OF — Two feeds, which is what the original interface specification froze.

WHAT BROKE — Measured on a wallet whose entire funding arrived from the Tornado Cash 0.1 ETH pool: the funding was *completely invisible* in the standard feed. Adding internal traces took that trace from 4 mixer findings to 8. Without it the tool reports "the trail ends here" at an address that was demonstrably funded by a mixer.

A token transfer amount is decoded from exactly one 32-byte word; any other data length skips the log entirely.

WHY — A contract can emit the Transfer topic with arbitrary data. One word is always within uint256.

INSTEAD OF — Parsing the whole data field as a number.

WHAT BREAKS — A hostile contract emits a multi-word payload, the decoded value exceeds uint256, and the database insert aborts the entire batch — a value-corruption and denial-of-service vector in one.

A bare `0x…` address is reported as *ambiguous*, never resolved to a guess.

WHY — Ethereum and Polygon share an address space. The registry returns every chain that claims the string and the caller must resolve it.

INSTEAD OF — Returning a single best guess and logging the assumption.

WHAT BREAKS — A confident trace of the wrong chain is a confident claim about the wrong money, and nobody reads the log line.

Movement identity is content-addressed, not positional. Each adapter supplies a `dedup_key` drawn from intrinsic structure — a Bitcoin input's vin position, a token

transfer's sender-recipient-contract triple. This is load-bearing: the first design numbered movements by a running counter, so the same transaction fetched while expanding address A and again while expanding address B produced different identities for the same transfer. The fact store then either silently dropped a real transfer or double-counted it. Both failures are now regression-tested by name.

> **Refusals are explicit.** An unconfirmed transaction is refused at normalization on every chain — it has no temporal anchor and can be replaced. A failed transaction moves no value, checked per chain in that chain's own idiom. An undeclared capability raises an error rather than returning empty data.

03

Persistence — making illegal states unrepresentable

Twelve tables in two planes: a global immutable fact store, and a per-investigation overlay that deletes with its case.

The split matters for an evidentiary reason. Facts — addresses, transactions, movements, assets, cached vendor payloads — are written once, shared across every investigation, and never rewritten. The overlay — investigations, nodes, edges, findings, evidence — belongs to one case and cascade-deletes with it. The labels and their audit log form a third, separate plane that neither owns.

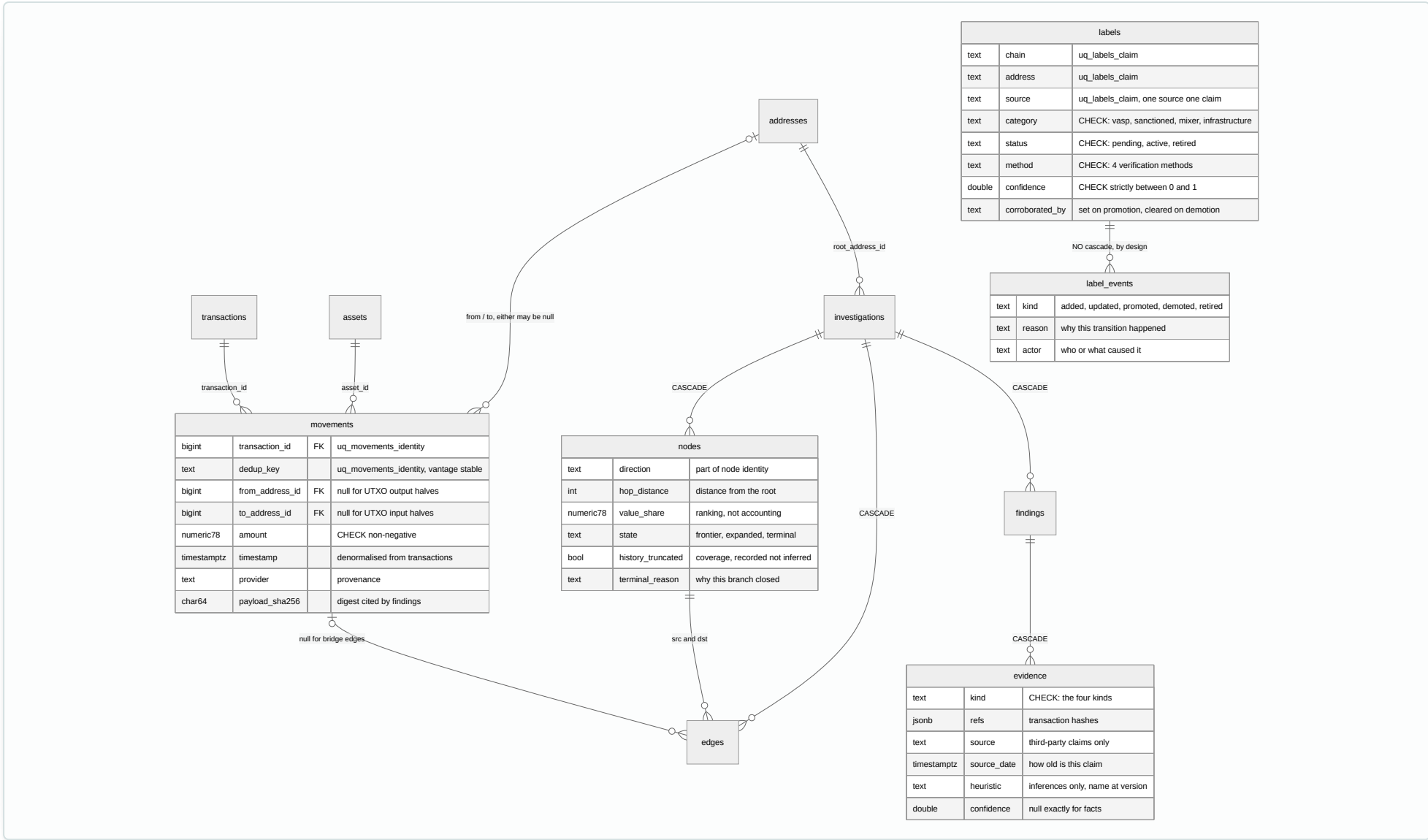

FIG 3 Abbreviated schema. 16 CHECK constraints, 7 unique constraints and 16 foreign keys hold invariants the application would otherwise have to remember.

Amounts are `NUMERIC(78,0)` surfaced as Python integers. Floats never touch value.

WHY — 78 digits holds a uint256. Value arithmetic is exact end to end.

INSTEAD OF — BIGINT or double precision.

WHAT BREAKS — BIGINT tops out around 9.2×10^{18} and a token balance overflows it outright. A float silently rounds wei-scale values, so two different transfers compare equal and a ranking becomes non-reproducible.

Read ordering breaks ties on chain data — timestamp, then transaction hash, then position — never on the database row id.

WHY — A row id is assigned at insert time, so ordering by it makes results depend on the sequence in which this database happened to ingest transactions.

INSTEAD OF — `ORDER BY timestamp, id` — the obvious stable-sort choice.

WHAT BREAKS — Sweep detection consumes each candidate as it pairs, so one reordered tie changes every later pairing. The same address would produce different findings against a warm database than against a cold one.

Direction is part of a node's identity.

WHY — An address reachable both backward and forward occupies two distinct trace positions, one per objective.

INSTEAD OF — Identity on (investigation, kind, address, transaction) alone, which is what the first migration created.

WHAT BROKE — The second objective's trace terminated at the first objective's node, and the engine reported "trace exhausted" for an endpoint it had already found. Escalated to critical in review because it makes the headline answer wrong without saying so.

The label audit table's foreign key has no cascade, while every other overlay key does.

WHY	Labels retire, they are never deleted. A cascade would let deleting a label silently delete its history — the one thing an audit table exists to prevent.
INSTEAD OF	`ON DELETE CASCADE`, matching the rest of the schema.
HOW IT IS HELD	A test issues a raw `DELETE FROM labels` and expects the database to reject it, specifically so that a future cascade — which no insert-path test would ever notice — fails here.

Migrations are a strictly linear chain of four Alembic revisions, autogenerated against the models and then hand-corrected. One downgrade is worth reading: the revision that added the fourth evidence kind refuses to run backwards while any row uses it, raising an error rather than rewriting immutable investigation records.

04 Label intelligence — what may be named, and why

The system holds 74,928 labels. Only rows in one specific state can ever put a name in a report.

Naming an address is the single highest-consequence thing this system does. "These funds reached Binance" sends an investigator to a real company with a real subpoena. Getting it wrong wastes a warrant, or worse, accuses the wrong business.

So a label is not a fact in a file. It is a claim with a lifecycle, and only the `active` state reaches the attributor. Pending and retired rows are not *filtered out* by the attributor — they simply never reach it.

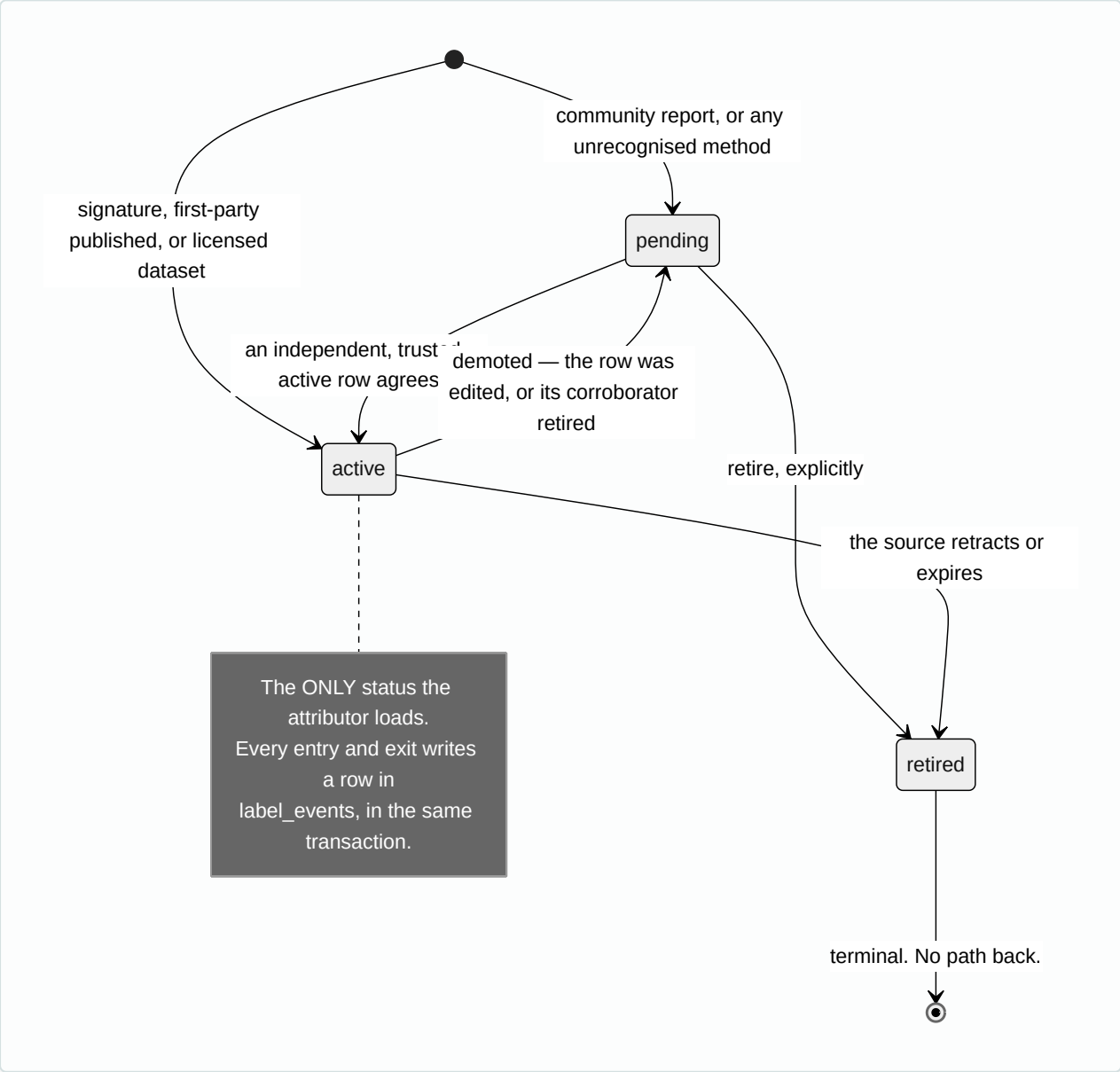

FIG 4 Verification method decides arrival status. Everything unrecognised fails closed to `pending`.

What is actually in the store today

SOURCE	METHOD	CONFIDENCE	ROWS	WHAT IT PROVES
OKX proof-of-reserves	`signature`	`0.90`	`36,049`	The exchange signed each address with its private key. This repo re-verified every signature itself.
OFAC SDN + official protocol sources	`first_party_published`	`0.90`	`70`	Sanctioned mixer addresses, published by the issuing authority.
Etherscan tags, via eth-labels	`licensed_dataset`	`0.75`	`25,923`	15 exchanges. Rests on Etherscan's private methodology — nobody outside can check it.
Etherscan tags — infrastructure	`licensed_dataset`	`0.75`	`12,886`	Known non-custodial contracts. Not an accusation — a veto (see layer 05).

EXCHANGE COVERAGE, BY ENTITY

ENTITY	OPERATIONAL	DEPOSIT	TOTAL	TIER
OKX	—	—	36,049	SIGNATURE-VERIFIED
Bitget	46	19,027	19,073	licensed dataset
Binance	199	5,021	5,220	licensed dataset
Coinbase	481	0	481	licensed dataset
Kraken	263	0	263	licensed dataset
HTX	232	0	232	licensed dataset
KuCoin	81	1	82	licensed dataset
Crypto.com	73	0	73	licensed dataset
Bitstamp	63	0	63	licensed dataset
Bybit	51	0	51	licensed dataset
Bitfinex, MEXC, Gemini, Gate.io, Huobi	108	0	108	licensed dataset

The distinction between **operational** and **deposit** addresses is legally material and is preserved in every finding. Funds arriving at a deposit address reached *a customer of that exchange*; funds arriving at an operational address reached the exchange's own treasury. Those are different claims and different requests to the exchange.

> **Why OKX dominates.** Not because OKX matters more, but because it is the one major exchange publishing a machine-readable, cryptographically signed address list. Binance and Coinbase are covered from a commercial tag dataset, which is weaker and Ethereum-only. Extending signature-grade coverage to a second exchange is the next planned source adapter; the verifier itself already handles ECDSA on EVM, Tron and Bitcoin multisig, and ed25519 on Solana.

The lifecycle rules, and the attacks that shaped them

Each of the following exists because adversarial review demonstrated the failure end-to-end against an earlier version of this layer. All four are now regression tests.

Activation is re-derived every cycle and synchronously on edit — never a stamp written once.

WHY	A claim is active because something still justifies it, not because something once did.
INSTEAD OF	Writing `corroborated_by` at promotion and trusting it thereafter.
DEMONSTRATED ATTACK	Submit honest content, wait for promotion, then edit the row to read "Lazarus Group / sanctioned". It stayed active with the original corroboration — attacker-chosen content wearing forged provenance, in the attributor's load.

A promoted community report can never itself corroborate another report.

WHY	Trust flows one way: from harvested sources to reports, never between reports.
INSTEAD OF	Treating "active" as sufficient to corroborate, since a promoted report has by definition already been checked.
DEMONSTRATED ATTACK	Two reports that could never promote each other directly did so across cycles once one had been promoted — and then held each other active after the real source retired.

Entity agreement is exact equality of a normalised stem, checked in both directions, with an empty stem matching nothing.

WHY	"Binance 14" confirms "Binance (deposit)". "Binance Charity" does not confirm "Binance" — different stem, plausibly a different legal entity, so it waits.
INSTEAD OF	Prefix or substring matching, which reads as the generous, obviously-safe relaxation.
WHAT BREAKS	A Binance Charity label would corroborate a report naming Binance. Separately, the index-stripping rule requires a separator before the digits — so "Binance 14" sheds its wallet number but "Aave V2" keeps its 2, and an Aave V2 pool never corroborates a claim about the V3 pool.

A community-submitted entity name must be a bare name: single line, no parentheses, no URLs, at most 64 characters.

WHY	Stem matching deliberately discards parentheticals — safe for our own curated packs, where the parenthetical *is* the role annotation.
INSTEAD OF	The same lenient rules for strangers as for curated packs, relying on corroboration to filter bad content.
DEMONSTRATED ATTACK	"Binance (successor wallet 0xATTACKER)" stems to "binance", corroborates against real Binance data, promotes to active — and the verbatim attacker prose becomes what an investigator reads as a sourced label. Corroboration cannot catch it, because the stem genuinely matches.

Corroboration errors are asymmetric, and the rules sit on the conservative side of every line. A false negative delays a true label by one cycle. A false positive publishes a stranger's assertion, under a citable source, to an officer deciding where to send a subpoena.

Every transition writes its event in the same database transaction as the row change — pinned by a fault-injection test that makes the event write fail and asserts the promotion did not survive. A promotion the audit table cannot show is a promotion nobody can audit.

One more property matters for provenance: the source cited in a report is always the *original* source — the proof-of-reserves file, the dataset, the corroborated report — never "the label store". The store is plumbing, not provenance.

05

Attribution and heuristics — a name and a guess, never confused

Two independent ways to conclude something about an address, kept structurally separate all the way to the wire.

Attribution is a lookup: does a sourced, active label claim this address? Heuristics are inference: does this address *behave* like something? The system runs both, and the type system makes it impossible to present the second as the first.

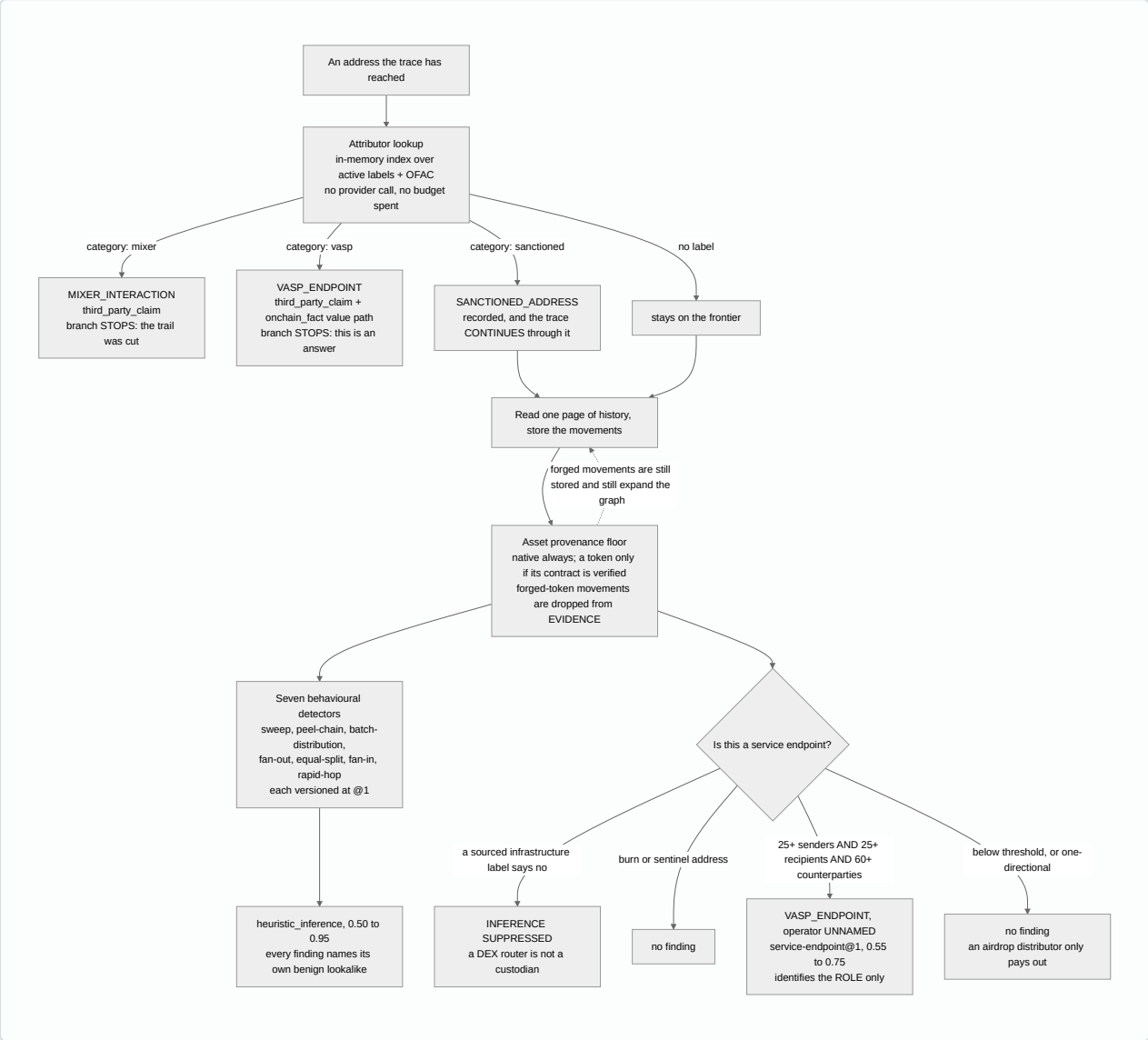

FIG 5 Four separate gates prevent a non-custodial contract from being reported as an exchange.

The four false-positive gates

1. **A sourced infrastructure label vetoes the behavioural inference.** A DEX settlement contract has the counterparty degree of an exchange and none of the custody. This is what the 12,886 infrastructure labels are for — they are not accusations, they are a veto list.
2. **Bidirectionality is required.** An airdrop distributor pays out to thousands and receives from almost nobody. Requiring 25+ distinct senders *and* 25+ distinct recipients excludes that shape.
3. **The asset provenance floor runs before any detector.** A token contract can emit transfer events between addresses that never signed anything. Movements in unverified assets are dropped from the evidence a heuristic may rest on — while still being stored and still expanding the graph, because hiding real on-chain events would be its own dishonesty.
4. **Every detector names the benign behaviour that looks identical to it,** inside the finding itself. Asserted by a test called `test_finding_names_the_benign_lookalike`.

The behavioural inference is honest about its own limits in its own text: it reports "service endpoint (operator unnamed)" and states that it identifies the role only — the operator's identity is off-chain and requires a sourced label. The obfuscation detectors refuse to allege intent at all: "this address fans value out to many recipients" is supportable from the chain; "this address is laundering" is not. A test named `test_never_names_a_specific_mixing_protocol` locks that in.

Labels are read the moment an address is *discovered*, not when it is eventually processed.

WHY	Attribution is a dictionary lookup — no network call, no budget. A free question should not sit behind an expensive one.
INSTEAD OF	Attributing only when a node is claimed for expansion, which is what shipped first.
WHAT BROKE	Measured on a real trace: the run held sourced 0.9-confidence labels for Binance and two Coinbase addresses, sitting at hop 2 in state `frontier` — discovered, never read — while reporting a 61% behavioural guess as its answer to "where did the funds come from". Across runs, 58–62% guesses were being reported over unread exchange labels.

06

The investigation engine

Bidirectional traversal with four budgets, five stop conditions, and two end-states — both of which are findings, never silence.

The engine walks outward from the root address in both directions at once. `BACKWARD` reads incoming movements — where funds came from. `FORWARD` reads outgoing movements — where funds went. Each discovered address becomes a node one hop further out, tagged with the direction that found it.

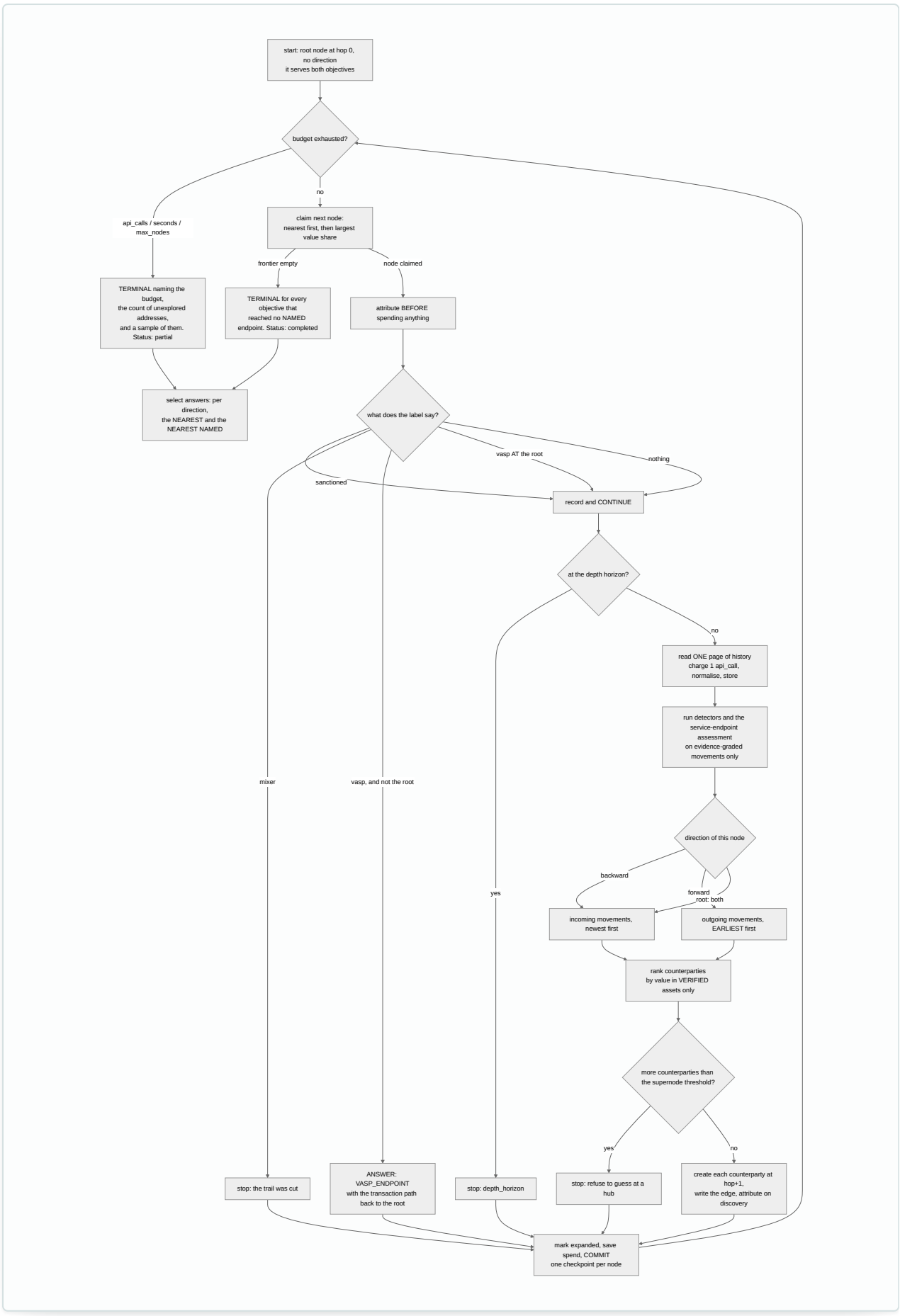

FIG 6 One committed checkpoint per node makes an investigation resumable, and makes a crash cost one node rather than a whole run.

Budgets

BUDGET	DEFAULT	PURPOSE
`api_calls`	`100`	One charge per address expansion (which is 3 upstream requests on Ethereum, 2 on Tron, 1 elsewhere)
`seconds`	`300`	Wall clock
`max_nodes`	`500`	Total addresses created
`max_depth`	6	Hops from the root. Enforced per node as a stop condition, not as exhaustion

When a budget binds, the run does not simply stop — it files a finding naming *which* budget stopped it, how many addresses were left unexplored, and a sample of them. A resumed run carries prior spend forward, so a crash-and-resume cannot grant itself a fresh budget.

The five stop conditions

REASON	MEANING	IS IT A COVERAGE GAP?
`vasp`	An answer was found here	No — this is success
`mixer`	The trail was deliberately cut	No — following through would be de-anonymisation
`depth_horizon`	Reached the configured hop limit	Yes, and it is counted and reported
`supernode`	Too many counterparties to attribute meaningfully	Yes, and the finding cites the transactions
`service_endpoint`	Behaves like custodial infrastructure, operator unnamed	Partly — it does not close the objective

Sanctions are deliberately not a stop condition. Funds moving *through* a sanctioned address are exactly what a trace must follow. Treating them as terminal would end the investigation at the hop an investigator most needs followed.

The frontier is claimed nearest-first, with value share only breaking ties inside a hop level.

WHY — The answer is phrased in hops — "nearest previous VASP, 3 hops away" — so the search claims in the order the goal is measured in. Under this ordering, a node's first-discovery hop *is* its minimum distance by construction.

INSTEAD OF — Claiming by value share first, which is what the frozen design document specified.

WHAT BREAKS — Without hop-relaxation machinery, a node's recorded hop is whatever path found it first — so "3 hops away" would be an upper bound presented as a fact. The accepted cost of the chosen order is stated openly: with a small budget a distant high-value cash-out may go unreached, and that surfaces as a *partial* run rather than a silently wrong number.

Backward reads newest-first; forward reads earliest-first. Behavioural analysis reads both sides newest-first through a separate query.

WHY — Backward tracing wants the most recent funding; forward tracing follows the cash-out, where the immediate post-arrival hops matter most.

INSTEAD OF — One uniform read order everywhere.

WHAT BREAKS — Uniform newest-first cuts the first cash-out hops off any busy forward node — the exact hops a forward trace exists to find. And a detector given the newest receipts but the *oldest* sends is comparing two windows that need not overlap at all: a sweep is a receipt followed by a forward, so it would find pairs that never happened and miss the ones that did.

Traversal ranking counts only assets whose provenance is established — but unverified-asset counterparties are still discovered, stored and explored.

WHY — Traversal order was steerable by anyone willing to pay gas.

INSTEAD OF — Ranking on everything (the original behaviour), or excluding unverified assets from the graph entirely.

WHAT BROKE — Measured on a live trace of the Bybit exploiter: the five highest-ranked unexplored branches were all worthless tokens at nominal values around 10^{26}, while the largest genuine movement was 3.22×10^{20} wei. Six counterfeit contracts each named the identical 1.394×10^{42} amount. The real trail sat below the spam until the budget ran out. Deleting those movements instead would erase the attacker's own footprints from the evidence record — so they are kept, ranked at zero, and the coverage statement says the ordering basis.

An objective is "answered" only by a VASP finding carrying a third-party claim. A behavioural inference never suppresses the honest terminal.

WHY — A finding earns "answered" by being citable. An inference stands *beside* the terminal, not instead of it — and the terminal's wording changes to say the operator is unnamed.

INSTEAD OF — Letting any VASP-shaped finding close the objective.

WHAT BROKE — That shipped. An inference reading "custodial infrastructure, operator unnamed, 61%" silently deleted the "trace exhausted without an attributed endpoint" terminal — turning a failure to name anyone into a confident-looking answer.

At an equal hop distance, the *unnamed* candidate wins the tie-break — deliberately preferring the weaker-looking finding.

WHY	A guess and a sourced label tied at the same hop are two different addresses. Preferring the label puts it in *both* slots, collapsing the pair to one row. Preferring the unnamed one costs nothing, because the named slot is about to show the label anyway.
INSTEAD OF	Preferring the named finding at a tie — the intuitive choice — or breaking ties by confidence.
WHAT BREAKS	A same-hop 62% guess vanishes from the report. The two slots collapse to one row only on object identity — literally the same finding — never on equal-looking fields.

Both end-states produce a **coverage statement** built by database query, not by counters held in memory — so a resumed run reports the same numbers the original did. It states how many transactions were examined, how many addresses had their history cut short, how many were left beyond the depth horizon, and how much of the frontier was never expanded. Every conclusion in the report cites it.

07

HTTP surface

Five routes, plus two that appear only when their dependency exists. The API owns no investigation behaviour of its own.

METHOD	PATH	RETURNS
GET	/healthz	Liveness
POST	/investigations	Creates a case, launches the run in the background, returns the id and the chain that was resolved
GET	/investigations/{id}	Status, budgets, spend, engine and ruleset versions
GET	/investigations/{id}/findings	Every finding with its evidence, plus the computed answers per direction
GET	/investigations/{id}/graph	Nodes and edges for rendering, with a truncation flag
GET	/metrics	Per-vendor success rates and latency percentiles — only when a provider pool exists
GET	/	The demo interface — only when the frontend file is present

Chain auto-detection may *eliminate* candidates on evidence, but never chooses between two chains that both hold history.

WHY — Probing can prove a chain holds nothing, which eliminates it. It cannot rank the chains that remain. Two live candidates return HTTP 422 with both named.

INSTEAD OF — Picking whichever chain returned more results and disclosing the guess in a log line.

WHAT BREAKS — The adapter merges several capped feeds, so every busy address looks equally full. The measured example: Binance 14 probes at 75 results on Polygon against 64 on Ethereum. That is noise, not evidence.

A provider failure during chain probing is reported as "could not check", never as "no transactions found".

WHY — Reporting "no transactions" would state a fact about the blockchain on the strength of an outage.

INSTEAD OF — Treating a probe exception as an empty result.

WHAT BREAKS — One live candidate plus one unreachable chain would auto-resolve to the live one, and a total outage would report "no history" — the tool confidently wrong about someone's money because a cache was down.

The graph read is capped per hop level, not just overall — and always reports the true total.

WHY — A truncated picture must keep every hop the trace reached.

INSTEAD OF — A single flat limit spent nearest-first.

WHAT BREAKS — Measured: on a live trace reaching hops −2 to +2, a flat cap of 120 returned only hops −1 to +1 and dropped every one of the 202 nodes at hop 2. The rendered picture lost a dimension without saying so, and read as a complete two-hop trace.

Amounts cross the wire as decimal *strings*, not JSON numbers.

WHY	Smallest-unit sums routinely exceed 2^{53} — a live trace carried 8.72×10^{29}.
INSTEAD OF	Emitting them as JSON integers.
WHAT BREAKS	JSON numbers land in a JavaScript double. The amount displayed on a forensic graph would not be the amount that moved.

Answer selection happens server-side, and both answers are always returned.

WHY	Every consumer that states "nearest previous VASP" must state the same thing.
INSTEAD OF	Returning raw findings and letting each client pick.
WHAT BREAKS	Which endpoint got reported would depend on traversal order per client. Reporting only the nearest hides a named exchange behind a behavioural guess; reporting only the named one drops the nearest.

There is no authentication on any route. This is a stated, deliberate gap, not an oversight. The demo runner binds to 127.0.0.1 precisely because of it, with a comment saying so. A ruling on this project fixed the sequence: *no public-facing surface ships before a minimal API-key layer exists* — operator-issued keys checked against hashes, per-key write rate limits, and every submitted report attributable to a key. The community-report endpoint and the case dashboard are both blocked behind that work.

One request, end to end

The path a single investigation takes, naming the real component at every boundary.

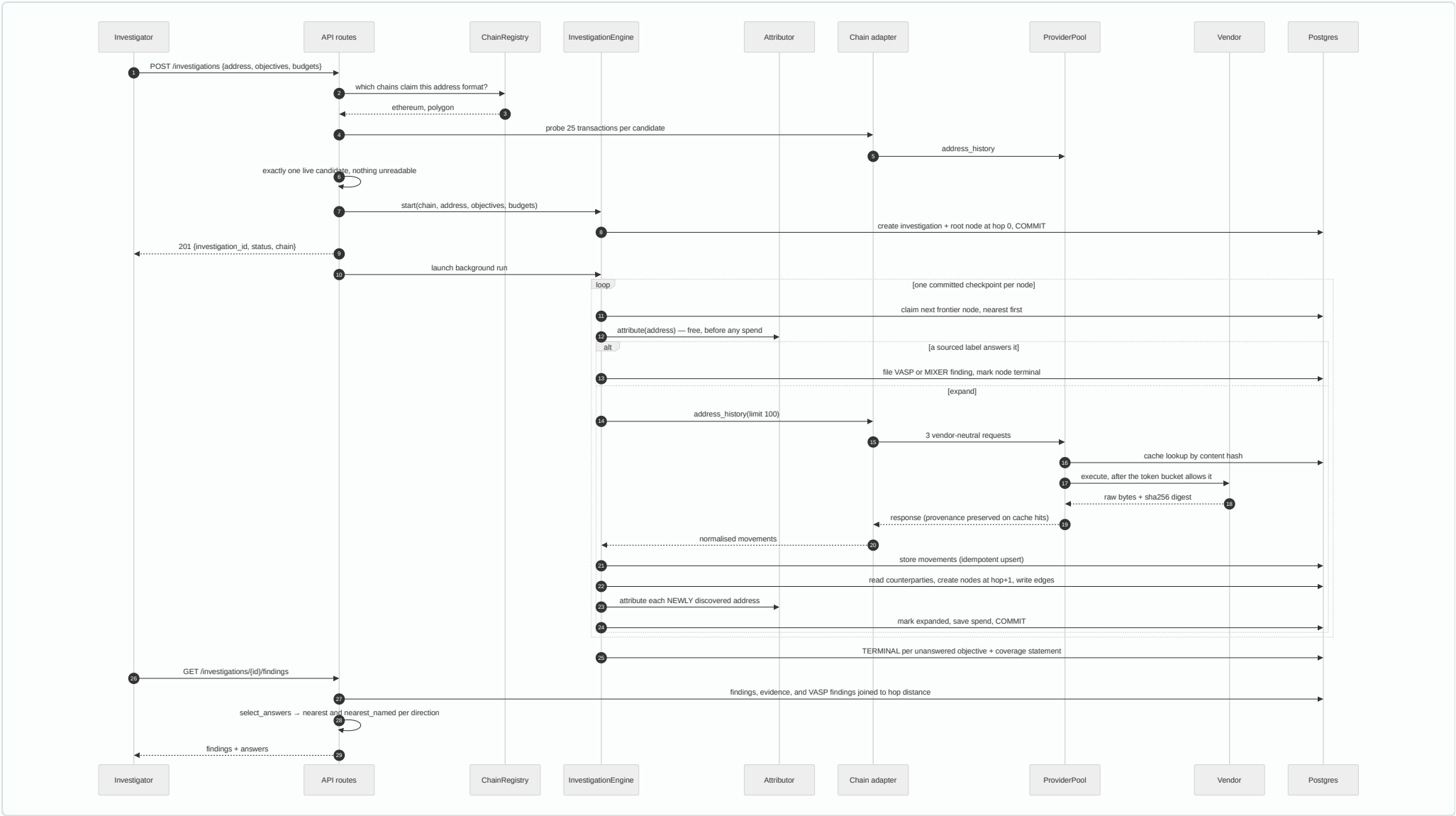

FIG 7 Note step ordering: the free label lookup always precedes any paid vendor call, and the response is returned to the investigator before the run completes.

1. **Validation.** Pydantic rejects an empty objective list and out-of-range budgets before any handler code runs.
2. **Chain resolution, once.** Resolving again after the investigation row is persisted could fail mid-request and orphan a case nothing will ever run. So it happens first, or not at all.
3. **Creation.** The address is canonicalised by the adapter — the engine must not know that "0x means lowercase" — then the investigation and its root node are written and committed in one transaction.
4. **Detach.** The run becomes a background task held by a strong reference. Without that reference the event loop keeps only a weak one, and a long investigation can be garbage-collected mid-flight, leaving a row stuck in "running" with no error.
5. **The loop.** Claim, attribute, guard, acquire, normalise, store, analyse, expand, commit. One checkpoint per node.
6. **The end state.** Either the frontier ran dry (completed) or a budget bound (partial). Both file findings; neither is silence.
7. **Read back.** Findings are rebuilt from rows, VASP findings are joined to their hop distance, and the two answers per direction are computed server-side.

Why this stack

Every dependency here was chosen against a specific alternative, for a reason that shows up in the evidence.

CHOICE	WHY IT	INSTEAD OF
Python 3.12+	The blockchain, cryptography and data ecosystem lives here. Signature verification across four curve/encoding combinations was possible without writing any crypto.	Go or Rust — faster, but the work is I/O-bound on rate-limited vendor APIs, so concurrency matters far more than raw speed.
FastAPI	Async-native, and its Pydantic models double as the validation layer *and* the OpenAPI schema. A malformed request is rejected before any handler runs. The API documents itself at `/docs`, which matters when a reviewer wants to see the contract.	Flask or Django. Django brings an ORM and admin this project does not want; Flask's async story is retrofitted.
PostgreSQL 16	`NUMERIC(78,0)` holds a uint256 exactly. Partial and expression indexes, JSONB for evidence references, and CHECK constraints strong enough to make illegal states unrepresentable at the storage layer.	SQLite (no NUMERIC of that precision, weak concurrency) or MongoDB (no constraints — every invariant would live only in application code).
SQLAlchemy 2.0, async	Typed declarative models that mypy checks in strict mode, with hand-written SQL available where the query matters. Sessions are per-unit-of-work; there is no global session.	Raw asyncpg — every constraint and join hand-maintained. Or the Django ORM, which is not async-native.
Alembic	Migrations autogenerate by diffing against the models, under a naming convention that makes the output deterministic. The test suite applies them to a throwaway database, so the migrations themselves are tested.	Hand-written SQL migrations, or schema-sync-on-startup — which cannot express the two downgrades in this project that must *refuse* to run rather than corrupt records.
httpx	One async client, shared across the process, with per-request timeouts and a mock transport that lets recorded vendor payloads be replayed through the *real* client and pool.	requests (synchronous) or aiohttp (no equivalent test transport).

CHOICE	WHY IT	INSTEAD OF
pytest, async by default	352 tests. Storage and engine tests run against a real Postgres 16 container that the fixture starts, migrates and truncates per test.	SQLite or mocks for the database — which would test a different database than the one that ships, and would not exercise the CHECK constraints that hold half the invariants.
ruff + mypy --strict	Eight lint rule families at 100 columns, and strict typing across the whole package. Run after every change.	black + flake8 + isort as three tools, and optional typing — under which the frozen domain model's guarantees would be advisory.

> **There is no machine learning in this system, deliberately.** Every conclusion must be explainable to a court. A model that outputs "87% likely to be an exchange" cannot be cross-examined; a rule that says "25 or more distinct senders and 25 or more distinct recipients and 60 or more total counterparties, version 1" can. Detectors are versioned so that the same inputs and the same version reproduce the same findings.

How it is verified

AREA	TESTS	WHAT THEY HOLD
chains	93	Every adapter against recorded real vendor payloads, with expectations recomputed from the fixture
analysis	92	Detector thresholds, the benign-lookalike requirement, infrastructure veto, OFAC screening
intel	45	Every lifecycle rule, and four demonstrated attacks as regression tests
storage	29	Constraints, migrations, movement identity, audit-log immutability
providers	25	The pipeline as an executable specification: cache-first, failover, breaker, 429 policy
investigation	22	Traversal, terminators, the tie-break, budget exhaustion
core	21	What must not be constructible, isn't
api	17	Full lifecycle over HTTP, chain-resolution refusals, graph truncation
runtime & regression	8	Composition root, plus specific review findings by name
Total	**352**	Passing, against a real Postgres 16 container, in 33 seconds

Two testing practices are worth naming to a reviewer, because they are the reason several of the defects in this document were found before they reached a report.

Adversarial review. Each subsystem is reviewed before commit by independent agents working from different angles, whose findings are then *refuted* by other agents rather than accepted. Three rounds on this codebase produced 23 findings, including three attacks demonstrated end-to-end against the label lifecycle. Every one is now a named regression test.

Mutation testing. Rules are deliberately broken to see whether the suite notices. This found tests that passed against code with the rule removed — for instance, loading pending labels into the attributor passed the entire suite, and a quiet fallback to reading label files when the store was empty also passed. Both are now pinned at the production wiring point, not at a helper.

The document produced from this codebase went through the same process: eight agents read one subsystem each from source, and two more cross-checked their output against the code. That pass caught ten factual errors before publication — an invented class name, and two claims that endpoints exist which do not.

Limits and refusals

Stated plainly, because a tool that hides its limits is more dangerous than one that has them.

What it refuses to do, by design

- **It will not follow value through a mixer.** That is de-anonymisation, out of scope, and the finding says the trail was *cut* rather than that it ran dry.
- **It will not name an operator from behaviour.** A behavioural inference reports the role only and stands beside the honest "no named endpoint" terminal, never instead of it.
- **It will not choose between two chains that both hold history.** It stops with a 422 and names both.
- **It will not report a provider outage as an empty ledger.**
- **It will not let an unverified claim name anyone,** at any confidence.
- **It ships no bridge address and no VASP label it has not verified.** The bridge registry is empty by default.

Known gaps

GAP	EFFECT	STATUS
No authentication anywhere in the API	Anyone who can reach the port can start cases and read findings	Blocking all public surface; API-key layer scoped and approved
The reconcile cycle has no scheduler	Promotions and demotions only happen when an operator runs them; a demotion does not reach a running process until restart	Harvester worker is the next build step
Solana has zero labels	Solana traces run but can never name an endpoint	Verifier built and tested; blocked on obtaining the source file
Tron history does not paginate	A busy Tron address returns one page and is recorded as fully read	Open — the coverage statement cannot flag it there
Detectors see at most 500 stored movements per address	A busy address may be judged on a window, and the coverage statement does not say when that cap bound	Open
Raw vendor payloads live in a TTL cache, not an archive	A cited digest may no longer resolve to stored bytes weeks later	Open — see the auditor's second question
Bridge crossings are detected, never followed	Cross-chain matching is a separate problem needing two chains at once	Deliberately deferred
No formal report output (PDF/DOCX) yet	Findings are available over the API and in the interface, not as a filed document	The largest remaining gap against the original goal

The auditor's questions

Nine attacks on this system's forensic soundness, each with the honest answer the code supports.

"Your evidence is forgeable. I deploy a worthless token, emit a transfer to your victim and another from them to an exchange, and your tool manufactures a pattern against a target I chose — the victim signed nothing."

ANSWERED

An asset provenance floor runs before any detector: native assets always pass, tokens only if the contract is in a verified pack. Forged movements cannot reach a heuristic, and cannot steer the traversal ranking either. The fail-closed default accepts native assets only, so an incorrectly wired engine still cannot be fed a forged token. The movements are still stored and still expand the graph — deliberately, since hiding real events would erase the attacker's own footprints — and the graph marks every edge with whether its asset was verified.

"Your findings cite a hash of the exact vendor bytes so they can be replayed. Show me the bytes behind this six-week-old finding."

LARGELY UNANSWERED — THE MOST SIGNIFICANT OPEN WEAKNESS

The digest is stored on every movement, but the raw bytes live only in the provider cache — which is a cache keyed by request, not a content-addressed archive keyed by digest. Address-history entries expire after five minutes and are overwritten on the next fetch. The digest then verifies nothing. What does hold: cache hits preserve the original provider name and digest rather than restamping them, and canonical JSON makes digests order-independent. An immutable payload store is not built.

"Your coverage statement claims to name every limit that closed a branch. It doesn't mention that the detectors only ever saw 500 movements of an address that has 40,000."

NOT ANSWERED

A 500-row cap governs every read feeding counterparty derivation and the detectors, and nothing records when it bound. The coverage statement reports page-truncated histories, depth-horizon nodes and the unexplored frontier — and will append "no address was left partially read" while the detectors judged a busy exchange address on 500 of its rows. The machinery to report it exists; this input is not wired into it.

"You told a court this address is Binance. Who says so, when did they say it, and was that claim still standing on the day you ran the trace?"

MOSTLY ANSWERED, WITH ONE LIVE GAP

Naming can only come from a third-party claim carrying a source and a source date, enforced at construction. Confidence is strictly below 1.0 at three layers including a database constraint. Only active rows reach the attributor, every transition writes an audit row in the same transaction, and the audit log cannot be deleted as a side effect. Corroboration is re-derived each cycle rather than stamped. **The gap:** the reconcile cycle has no production caller — "continuously justified" is a property of a scheduler that does not yet exist in this repository. Second gap: findings record the label's source string but not the label row id, so a report cannot be bound back to that row's exact event history.

"A bare 0x address exists on Ethereum and Polygon. You picked one. How do you know you traced the right money?"

ANSWERED — AND THE REFUSAL *IS* THE ANSWER

Each candidate is probed with one cheap call; chains holding nothing are eliminated. Resolution happens only when exactly one live candidate remains *and* nothing was unreadable. Two live candidates raise a structured error rather than guessing. A chain that could not be probed deliberately poisons the result, because one live candidate plus a chain we could not read is still not one candidate.

"Your report says 'nearest next VASP: service endpoint' — that's a guess about an address, dressed as an endpoint an investigator can act on."

ANSWERED

The heuristic says "operator unnamed" in its own summary, carries a versioned inference capped at 0.75, and cannot close the objective — that requires a citable third-party claim, so the honest "trace exhausted without a named endpoint" terminal is filed beside it. A sourced infrastructure label vetoes the inference outright, and burn addresses are excluded. The two answer slots are surfaced separately and refuse to collapse.

"Your sanctions screening returned clean. Was the list actually loaded, and how old is it?"

HALF ANSWERED

A missing dataset logs a warning naming the chain whose screening is disabled — deliberately not silent — but nothing about that degradation reaches the investigation record. A reader cannot distinguish a clean screen from an unloaded list. Staleness *is* stamped into every claim: the snapshot date rides on the evidence to the wire. But the refresh script the module's own documentation names does not exist yet, so the snapshot is fixed with no shipped way to update it.

"Two runs over the same stored data produced two different reports — which one do I file?"

ANSWERED AT THE LEVEL THE CODE CONTROLS

Ordering ties break on chain data, never on database row ids. Evidence references are sorted before truncation everywhere. Movement identity is vantage-stable under a uniqueness constraint, so re-reading a transaction from a different feed deduplicates rather than duplicating. A resumed run carries prior spend forward. Residual non-determinism the code does not remove: elapsed seconds are per-run by design, and detector inputs depend on the 500-row window, which depends on what other investigations have written into the shared fact store.

"Who else could have started, altered or read this case? Show me the chain of custody on the case file itself."

NOT ANSWERED, AND DISCLOSED

There is no authentication, authorisation or rate limiting anywhere in the API. The only control is the loopback bind in the demo runner, with a comment saying exactly why. Partial mitigations that do exist: the HTTP client loggers are raised to WARNING specifically so provider credentials in URLs never land in the retained log, and every log line carries the bound investigation id, making the application log a per-investigation trail. Closing this is the next build item and blocks everything public-facing.

FATHOM 0.1.0 · backend technical reference · compiled 13 August 2026 from branch feat/backend-v1

Every symbol name, count and threshold in this document was read from the source at compile time. Where this document and the code disagree, the code is right.

This system is decision support. It makes no claim of court admissibility or legal certification.